

Documentación

Problema:

El proyecto desarrollado consiste en una aplicación de consola para la gestión de tareas. El objetivo principal es permitir al usuario añadir, listar, buscar y eliminar tareas mediante un menú interactivo.

Cada tarea contiene información básica como un identificador, un título, una prioridad y un estado de completado. Además, el proyecto implementa funcionalidades modernas de Java como colecciones, programación genérica, uso de Optional y la API Stream.

La aplicación está estructurada siguiendo una separación básica en capas:

Vista: interacción con el usuario mediante consola.

Modelo: clases que representan las entidades del sistema.

Controlador/Servicio: lógica de negocio y gestión de datos.

Estructuras de datos utilizadas

La colección **ArrayList** se emplea para almacenar dinámicamente las tareas creadas por el usuario.

Razón:

- Permite almacenar elementos de forma dinámica.
- Facilita recorridos rápidos mediante iteraciones y Streams.
- Resulta adecuada para operaciones frecuentes de lectura y listado.
- Su implementación es sencilla y eficiente para aplicaciones pequeñas o medianas.

Estos métodos usados han sido:

- **add()** para insertar tareas.
- **removeIf()** para eliminar.
- **forEach()** para recorrer elementos.

Uso de genéricos

El proyecto utiliza programación genérica mediante la clase:

public class Repositorio<T>

Los genéricos permiten reutilizar código para trabajar con distintos tipos de datos sin duplicar clases.

Repositorio<Tarea>
Repositorio<Usuario>

Gracias a esto, la clase Repositorio puede almacenar cualquier tipo de objeto manteniendo seguridad de tipos durante la compilación.

Uso de Optional

La clase Optional se utiliza en el método de búsqueda de tareas:

public Optional<Tarea> buscarPorId(int id)

Su finalidad es evitar valores nulos y prevenir errores como NullPointerException.

El resultado se maneja con:

ifPresentOrElse()

Esto permite:

- Mostrar la tarea si existe.
- Mostrar un mensaje alternativo si no se encuentra.

Uso de Stream

La API Stream se emplea para realizar operaciones funcionales sobre la colección de tareas.

Ejemplos implementados:

Filtrado

- ***filter()***

Obtiene únicamente tareas pendientes.

Transformación

- ***map()***

Extrae los títulos de las tareas.

Conteo

- ***count()***

Cuenta las tareas completadas.

Ordenación

- *sorted()*

Ordena las tareas según prioridad.

MUESTRAS DE SALIDA:

Crear y mostrar

```
1. Añadir tarea
2. Listar tareas
3. Buscar tarea
4. Eliminar tarea
5. Completar tarea
6. Salir
Opción: 1
ID: 1
Título: Libro 1
Prioridad: 2

1. Añadir tarea
2. Listar tareas
3. Buscar tarea
4. Eliminar tarea
5. Completar tarea
6. Salir
Opción: 1
ID: 2
Título: Libro 2
Prioridad: 1

1. Añadir tarea
2. Listar tareas
3. Buscar tarea
4. Eliminar tarea
5. Completar tarea
6. Salir
Opción: 2
1 - Libro 1 (Prioridad: 2) [PENDIENTE]
2 - Libro 2 (Prioridad: 1) [PENDIENTE]
```

Filtrar - Búsqueda

```
Opción: 3
ID a buscar: 2
2 - Libro 2 (Prioridad: 1) [PENDIENTE]
```

Actualizar

```
Opción: 5
ID de la tarea completada: 2
Tarea completada.
```

```
Opción: 2
1 - Libro 1 (Prioridad: 2) [PENDIENTE]
2 - Libro 2 (Prioridad: 1) [COMPLETADA]
```

Borrar

```
Opción: 4
ID a eliminar: 1
```

```
Opción: 2
2 - Libro 2 (Prioridad: 1) [COMPLETADA]

1. Añadir tarea
2. Listar tareas
```